EXPERIMENT-06

Aim: Implementing broadcast receivers. Design a broadcast receiver to respond to system or application-level broadcast. Test the receiver with different broadcast scenarios.

A Broadcast Receiver is an Android component that listens for broadcast messages sent by the Android system or other applications.

Theory:
A Broadcast Receiver is an Android component that listens for system or application-level broadcast messages. When a specific event occurs (like battery change or charger connected), the receiver automatically triggers the onReceive() method to perform an action.

1. Create Project
   - Open Android Studio
   - Click New Project → Empty Activity
   - Select language: Java
2. Create Broadcast Receiver Class
   - Right-click package → New → Java Class → MyReceiver
   - Extend BroadcastReceiver
   - Override onReceive() method
   - Show Toast message when broadcast is received
3. Register Receiver
   - Open AndroidManifest.xml
   - Add inside <application>:

```
<receiver android:name=".MyReceiver">
  <intent-filter>
    <action android:name="android.intent.action.ACTION_POWER_CONNECTED"/>
  </intent-filter>
</receiver>
```

4. Run Application
   - Connect emulator or device
   - Run the app
5. Test Broadcast Scenarios
   - Power Connected

- Plug charger → Toast message appears

- Power Disconnected
  - Change action in Manifest:

<action android:name="android.intent.action.ACTION_POWER_DISCONNECTED"/>

- Remove charger → Toast appears

Code:

MyReceiver.java

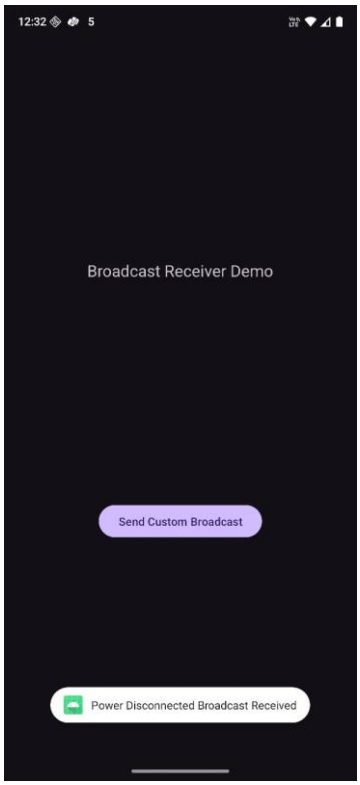
```java
package com.example.experiment_06;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context,
                text: "Power Connected Broadcast Received",
                Toast.LENGTH_LONG).show();
    }
}
```

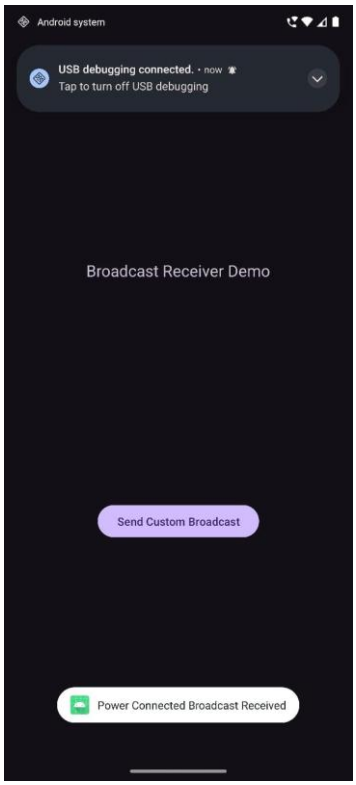
AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="Experiment-06"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Experiment06">

        <receiver
            android:name=".MyReceiver"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.ACTION_POWER_CONNECTED"/>
                <action android:name="android.intent.action.ACTION_POWER_DISCONNECTED"/>
            </intent-filter>
        </receiver>

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="Experiment-06"
            android:theme="@style/Theme.Experiment06">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Output:

1. System Broadcast Detection
   - App listens for system events like power connection
2. Automatic Trigger
   - When event occurs, onReceive() is executed automatically
3. Real-time Response
   - Displays Toast message when broadcast is received
4. Different Scenarios
   - Works for multiple events:
   - Power connected
   - Power disconnected
5. Background Functionality
   - Works even without user interaction





Conclusion:

This practical demonstrates how Broadcast Receivers handle system events and allow applications to respond automatically to changes in the device environment.